

SQL 기반 금융권 팀 프로젝트 보고서

1. 프로젝트 개요

- 조 이름: 박신양
- 프로젝트 이름: 함께하는 투자, 함.투
- 진행기간: 2025년 1월 31일 - 2025년 2월 3일

2. 프로젝트 팀원

- 박대균
- 신철환
- 양준성

3. 프로젝트 배경 및 목적

최근 주식에 대한 관심이 높아지면서 다양한 애플리케이션이 등장하고 있습니다. 하지만 너무 방대한 정보와 투명한 주식 커뮤니티는 없다고 느꼈고 이를 해결하기 위해 전문가의 의견을 얻을 수 있는 키움증권의 리서치 페이지와 사람들끼리 종목에 대한 정보를 교환할 수 있는 토스증권의 커뮤니티 페이지를 참고하여 주식에 대한 전문 지식과 의견을 나눌 수 있는 서비스를 기획했습니다.

4. 프로젝트 구조 및 단계별 진행 내용

4.1 데이터 수집

- 키움증권의 리서치 페이지

기업분석팀	투자전략팀	글로벌리서치팀	혁신성장리서치팀
-------	-------	---------	----------

국내외 주요 산업 및 기업 분석, 국내외 상장사 주가 전망 및 비상장사 가치평가, 기관 및 개인투자자 대상 업종/기업 설명회 등의 서비스를 제공하고 있습니다. 기본에 충실하면서도 시장에 기민하게 대처하면서 최고의 투자 아이디어들을 제공하고 있습니다.



박상준 수석연구위원/기업분석팀장

학력 성균관대 경영학

경력 NH-CA자산운용 7년

이메일 sjpark@kiwoom.com

#음식료 #유통

분석자료검색



김은갑 수석연구위원

학력 서울대 경영학

경력 IBK, KTB, BNK, PwC Consulting, NH투자증권, IBM 등

이메일 egab@kiwoom.com

#은행

분석자료검색

• 토스증권의 커뮤니티 페이지



미다스의손절 · 방금 전

이젠 모르겠다..본전만 되면 이사갈때 티비 침대 컴퓨터 바꿀했는데..그냥 계속 써야겠네

♡ 💬 ↗



다으니맘 종목 수집가 · 방금 전

애국하겠습니다.50600원에..매수..걸어둡...;;

♡ 💬 ↗



슈퍼맨처럼 · 1분 전

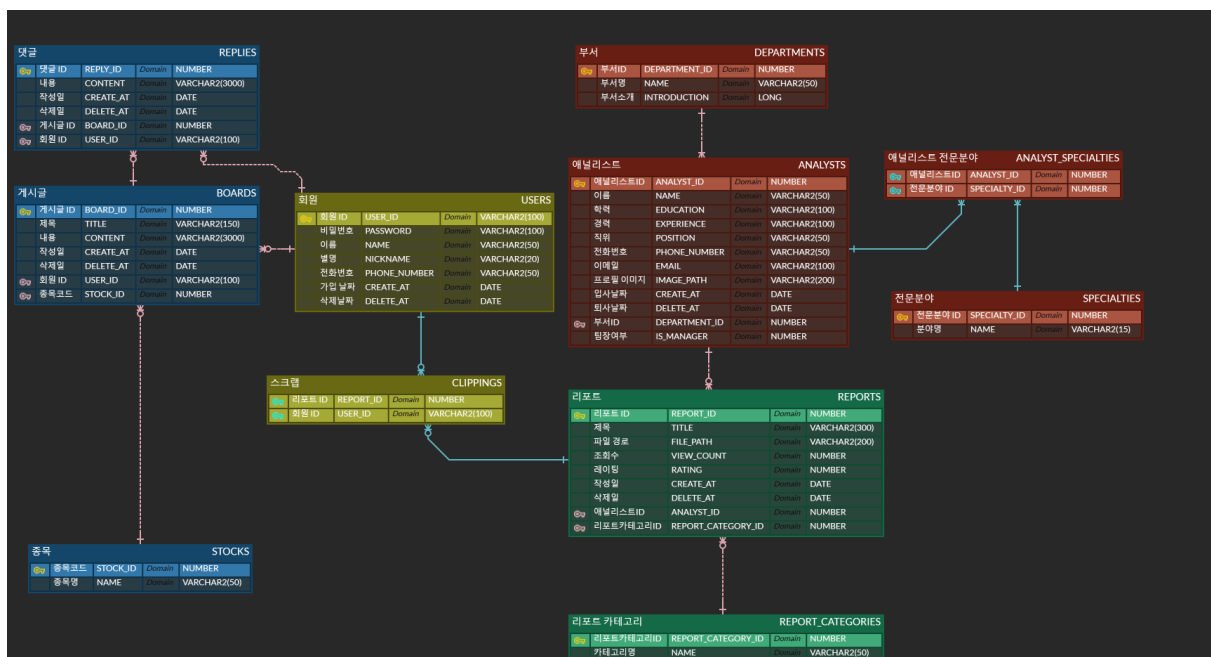
주주 #그만들좀 파세요~

♡ 💬 ↗

4.2 데이터베이스 설계

- 데이터베이스 시스템 : ORACLE
- 설계된 테이블
 - USERS (회원)
 - DEPARTMENTS (부서)
 - ANALYSTS (애널리스트)
 - SPECIALTIES (전문분야)
 - ANALYST_SPECIALIES (애널리스트 전문분야)

- REPORTS (리포트)
- REPORT_CATEGORIES (리포트 카테고리)
- CLIPPINGS (스크랩)
- STOCKS (종목)
- BOARDS (게시글)
- COMMENTS (댓글)
- 테이블 관계
 - DEPARTMENTS ↔ ANALYSTS (1 : N)
 - ANALYSTS ↔ ANALYST_SPECIALTIES ↔ SPECIALTIES (N : N)
 - ANALYSTS ↔ REPORTS (1 : N)
 - REPORTS ↔ REPORT_CATEGORIES (1 : N)
 - USERS ↔ CLIPPING ↔ REPORTS (N : N)
 - USERS ↔ BOARDS (1 : N)
 - USERS ↔ REPLIES (1 : N)
 - BOARDS ↔ REPLIES (1 : N)
 - STOCKS ↔ BOARDS (1 : N)



4.3 SQL 쿼리 설계 및 구현

-- 1. 게시글 작성

```
INSERT INTO BOARDS(BOARD_ID, TITLE, CONTENT, CREATE_AT, USER_ID, STOCK_ID)
VALUES(BOARDS_SEQUENCE.NEXTVAL, '이 주식 좋아요~', '무조건 오름', SYSDATE, 1, 1);
```

-- 2. 게시글 조회

-- 기본

```
SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY B.CREATE_AT DESC) AS IDX,
        B.BOARD_ID,
        CASE
            WHEN B.DELETE_AT IS NULL THEN B.TITLE
            ELSE '삭제한 게시글'
        END AS TITLE,
        CASE
            WHEN B.DELETE_AT IS NULL THEN B.CONTENT
            ELSE '-'
        END AS CONTENT,
        B.CREATE_AT,
        B.USER_ID,
        CASE
            WHEN U.DELETE_AT IS NULL THEN U.NICKNAME
            ELSE '탈퇴한 회원'
        END AS NICKNAME,
        B.STOCK_ID,
        S.NAME AS STOCK_NAME,
        (SELECT COUNT(*) FROM REPLIES R WHERE R.BOARD_ID = B.BOARD_ID) AS REPLY_COUNT
    FROM BOARDS B, USERS U, STOCKS S
    WHERE B.USER_ID = U.USER_ID
        AND B.STOCK_ID = S.STOCK_ID
)
WHERE IDX BETWEEN 11 AND 20;
```

-- 게시글 종목별 조회

```
SELECT *
FROM (
```

```

SELECT
    ROW_NUMBER() OVER (ORDER BY B.CREATE_AT DESC) AS IDX,
    B.BOARD_ID,
    CASE
        WHEN B.DELETE_AT IS NULL THEN B.TITLE
        ELSE '삭제한 게시글'
    END AS TITLE,
    CASE
        WHEN B.DELETE_AT IS NULL THEN B.CONTENT
        ELSE '-'
    END AS CONTENT,
    B.CREATE_AT,
    B.USER_ID,
    CASE
        WHEN U.DELETE_AT IS NULL THEN U.NICKNAME
        ELSE '탈퇴한 회원'
    END AS NICKNAME,
    B.STOCK_ID,
    S.NAME AS STOCK_NAME,
    (SELECT COUNT(*) FROM REPLIES R WHERE R.BOARD_ID = B.BOARD_ID)
FROM BOARDS B, USERS U, STOCKS S
WHERE B.USER_ID = U.USER_ID
    AND B.STOCK_ID = S.STOCK_ID
    AND B.STOCK_ID = 4
)
WHERE IDX BETWEEN 1 AND 10;

-- 게시글 유저별 조회
SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY B.CREATE_AT DESC) AS IDX,
        B.BOARD_ID,
        CASE
            WHEN B.DELETE_AT IS NULL THEN B.TITLE
            ELSE '삭제한 게시글'
        END AS TITLE,
        CASE

```

```

        WHEN B.DELETE_AT IS NULL THEN B.CONTENT
        ELSE '-'
    END AS CONTENT,
    B.CREATE_AT,
    B.USER_ID,
    CASE
        WHEN U.DELETE_AT IS NULL THEN U.NICKNAME
        ELSE '탈퇴한 회원'
    END AS NICKNAME,
    B.STOCK_ID,
    S.NAME AS STOCK_NAME,
    (SELECT COUNT(*) FROM REPLIES R WHERE R.BOARD_ID = B.BOARD_ID)
FROM BOARDS B, USERS U, STOCKS S
WHERE B.USER_ID = U.USER_ID
    AND B.STOCK_ID = S.STOCK_ID
    AND U.USER_ID = 2
)
WHERE IDX BETWEEN 1 AND 10;

```

-- 3. 게시글 하위 댓글 조회

```

SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY R.CREATE_AT) AS IDX,
        R.REPLY_ID,
        CASE
            WHEN R.DELETE_AT IS NULL THEN R.CONTENT
            ELSE '삭제한 댓글'
        END AS CONTENT,
        R.CREATE_AT,
        R.BOARD_ID,
        R.USER_ID,
        CASE
            WHEN U.DELETE_AT IS NULL THEN U.NICKNAME
            ELSE '탈퇴한 회원'
        END AS NICKNAME
    FROM REPLIES R, USERS U
)

```

```

WHERE R.USER_ID = U.USER_ID
      AND R.BOARD_ID = 5
)
WHERE IDX BETWEEN 1 AND 10;

-- 고객 관심사 파악
-- 1. 한달 동안 가장 많은 게시글이 만들어진 종목들

SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER(ORDER BY COUNT(*) DESC) AS IDX,
        S.STOCK_ID,
        S.NAME,
        COUNT(*) AS BOARD_COUNT
    FROM BOARDS B, STOCKS S
    WHERE B.STOCK_ID = S.STOCK_ID
        AND B.CREATE_AT BETWEEN SYSDATE - INTERVAL '1' MONTH AND SYS
    GROUP BY S.STOCK_ID, S.NAME
)
WHERE IDX <= 5;

-- 2. 한달 동안 가장 많은 댓글이 달린 게시글들
SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS IDX,
        R.BOARD_ID,
        COUNT(*) AS REPLY_COUNT
    FROM REPLIES R
    WHERE R.CREATE_AT BETWEEN SYSDATE - INTERVAL '1' MONTH AND SY
    GROUP BY R.BOARD_ID
)
WHERE IDX <= 5;

-- 3. 한달 동안 가장 많은 댓글이 달린 유저들

```

```

SELECT *
FROM (
    SELECT
        ROW_NUMBER() OVER (ORDER BY COUNT(*) DESC) AS IDX,
        U.USER_ID,
        U.NICKNAME,
        COUNT(*) AS REPLY_COUNT
    FROM USERS U, BOARDS B, REPLIES R
    WHERE U.USER_ID = B.USER_ID
        AND B.BOARD_ID = R.BOARD_ID
        AND R.CREATE_AT BETWEEN SYSDATE - INTERVAL '1' MONTH AND SYS
    GROUP BY U.USER_ID, U.NICKNAME
)
WHERE IDX <= 5;

```

-- 4.리포트 작성

```

INSERT INTO REPORTS VALUES (
    REPORTS_SEQUENCE.NEXTVAL,
    'TEST 1',
    '첨부파일_TEST',
    0,
    0,
    SYSDATE,
    NULL,
    1,
    1
);

```

-- 5.리포트 조회(필터 기능 포함)

-- 특정 분석가의 리포트 조회

```

SELECT RC.NAME CATEGORY, R.TITLE, R.FILE_PATH, R.VIEW_COUNT, R.RATI
FROM REPORTS R
JOIN REPORT_CATEGORIES RC
ON R.REPORT_CATEGORY_ID = RC.REPORT_CATEGORY_ID
WHERE R.ANALYST_ID = 6
ORDER BY R.CREATE_AT DESC;

```

-- RATING이 가장 높은 5개 선정 (같은 점수일 땐 VIEW_COUNT로 내림차순)


```

SELECT *
FROM (
    SELECT R.TITLE, A.NAME, R.RATING, R.VIEW_COUNT
    FROM REPORTS R, ANALYSTS A
    WHERE R.ANALYST_ID = A.ANALYST_ID
    ORDER BY RATING DESC, VIEW_COUNT DESC
)
WHERE ROWNUM <= 5;

-- VIEW_COUNT가 가장 높은 5개 선정 (같은 땐 RATING으로 내림차순)
SELECT *
FROM (
    SELECT R.TITLE, A.NAME, R.RATING, R.VIEW_COUNT
    FROM REPORTS R, ANALYSTS A
    WHERE R.ANALYST_ID = A.ANALYST_ID
    ORDER BY VIEW_COUNT DESC, RATING DESC
)
WHERE ROWNUM <= 5;

-- 평균 RATING이 가장 높은 애널리스트의 리포트 조회 (평점으로 내림차순)
SELECT RC.NAME CATEGORY, R.TITLE, A.NAME ANALYST, R.VIEW_COUNT, I
FROM REPORTS R, ANALYSTS A, REPORT_CATEGORIES RC
WHERE R.ANALYST_ID = A.ANALYST_ID
AND R.REPORT_CATEGORY_ID = RC.REPORT_CATEGORY_ID
AND R.ANALYST_ID = (
    SELECT ANALYST_ID
    FROM (
        SELECT ANALYST_ID, AVG(RATING) AVG_RATING
        FROM REPORTS
        GROUP BY ANALYST_ID
        ORDER BY AVG_RATING DESC
    )
    WHERE ROWNUM = 1)
ORDER BY RATING DESC;

-- 평균 VIEW_COUNT가 가장 높은 애널리스트의 리포트 조회 (조회수로 내림차순)
SELECT RC.NAME CATEGORY, R.TITLE, A.NAME ANALYST, R.VIEW_COUNT, I
FROM REPORTS R, ANALYSTS A, REPORT_CATEGORIES RC

```

```

WHERE R.ANALYST_ID = A.ANALYST_ID
AND R.REPORT_CATEGORY_ID = RC.REPORT_CATEGORY_ID
AND R.ANALYST_ID = (
    SELECT ANALYST_ID
    FROM (
        SELECT ANALYST_ID, AVG(VIEW_COUNT) AVG_VIEW_COUNT
        FROM REPORTS
        GROUP BY ANALYST_ID
        ORDER BY AVG_VIEW_COUNT DESC
    )
    WHERE ROWNUM = 1)
ORDER BY R.VIEW_COUNT DESC;

```

```

-- 부서 평가
-- 특정 기간동안 조회수가 가장 많은 글을 작성한 부서명
-- 이번 달 조회수가 가장 많은 글을 작성한 부서명

```

```

SELECT DEPARTMENT
FROM (
    SELECT D.NAME DEPARTMENT, R.VIEW_COUNT
    FROM DEPARTMENTS D, REPORTS R, ANALYSTS A
    WHERE D.DEPARTMENT_ID = A.DEPARTMENT_ID
    AND R.ANALYST_ID = A.ANALYST_ID
    AND R.CREATE_AT >= ADD_MONTHS(SYSDATE, -1)
    ORDER BY R.VIEW_COUNT DESC
)
WHERE ROWNUM = 1;

```

```

-- 올해 조회수가 가장 많은 글을 작성한 부서명
SELECT DEPARTMENT
FROM (
    SELECT D.NAME DEPARTMENT, R.VIEW_COUNT
    FROM DEPARTMENTS D, REPORTS R, ANALYSTS A
    WHERE D.DEPARTMENT_ID = A.DEPARTMENT_ID
    AND R.ANALYST_ID = A.ANALYST_ID
    AND R.CREATE_AT >= ADD_MONTHS(SYSDATE, -12)
    ORDER BY R.VIEW_COUNT DESC
)
WHERE ROWNUM = 1;

```

```

-- 부서별 년도별 총 조회수
SELECT D.NAME, TO_CHAR(R.CREATE_AT, 'YYYY') CREATE_YEAR, SUM(R.VIEW_COUNT) AS TOTAL_VIEW_COUNT
FROM DEPARTMENTS D, REPORTS R, ANALYSTS A
WHERE D.DEPARTMENT_ID = A.DEPARTMENT_ID
AND R.ANALYST_ID = A.ANALYST_ID
GROUP BY ROLLUP(D.NAME, TO_CHAR(R.CREATE_AT, 'YYYY'))
ORDER BY D.NAME, CREATE_YEAR;

-- 개인 평가
-- 분석가 쓴 포스트 중 평점이 가장 높은 알티클(평점이 같은 경우, 조회수가 높은 쪽으로)
SELECT
    A.NAME,
    T.TITLE AS BEST_REPORT_TITLE
FROM (
    SELECT
        R.REPORT_ID,
        R.TITLE,
        R.ANALYST_ID,
        R.RATING,
        R.VIEW_COUNT,
        ROW_NUMBER() OVER (
            PARTITION BY R.ANALYST_ID
            ORDER BY R.RATING DESC, R.VIEW_COUNT DESC
        ) AS RANK
    FROM REPORTS R
) T
, ANALYSTS A
WHERE T.ANALYST_ID = A.ANALYST_ID
AND T.RANK = 1;

-- 특정 분석가의 월별 리포트 작성량
SELECT A.NAME, TO_CHAR(R.CREATE_AT, 'YYYY-MM') CREATE_MONTH, COUNT(R.REPORT_ID) AS REPORT_COUNT
FROM ANALYSTS A
LEFT OUTER JOIN REPORTS R
ON A.ANALYST_ID = R.ANALYST_ID
GROUP BY ROLLUP(A.NAME, TO_CHAR(R.CREATE_AT, 'YYYY-MM'))
ORDER BY A.NAME, CREATE_MONTH NULLS LAST;

```

```

-- 특정 부서의 월별 리포트 작성량
SELECT D.NAME, TO_CHAR(R.CREATE_AT, 'YYYY-MM') CREATE_MONTH, CO
FROM DEPARTMENTS D
JOIN ANALYSTS A
ON D.DEPARTMENT_ID = A.DEPARTMENT_ID
LEFT OUTER JOIN REPORTS R
ON A.ANALYST_ID = R.ANALYST_ID
GROUP BY ROLLUP(D.NAME, TO_CHAR(R.CREATE_AT, 'YYYY-MM'))
ORDER BY D.NAME, CREATE_MONTH NULLS LAST;

-- 6. 스크랩 및 모아보기
INSERT INTO CLIPPINGS VALUES (5, 1);
INSERT INTO CLIPPINGS VALUES (5, 10);
INSERT INTO CLIPPINGS VALUES (5, 100);
INSERT INTO CLIPPINGS VALUES (5, 1000);

-- 3번 유저의 스크랩 모아보기
SELECT R.TITLE, A.NAME, R.FILE_PATH, R.RATING, R.VIEW_COUNT
FROM REPORTS R, CLIPPINGS C, ANALYSTS A
WHERE R.REPORT_ID = C.REPORT_ID
AND R.ANALYST_ID = A.ANALYST_ID
AND C.USER_ID = 3;

```

5. 프로젝트 결과

해당 프로젝트를 통해 실제 키움증권의 리서치 페이지와 토스증권의 커뮤니티 페이지를 참고하여 전문 애널리스트의 투자 리포트 저장 / 조회하고 투자자들끼리 조회할 수 있는 스키마와 쿼리를 구현했습니다.

6. 프로젝트의 의의 및 개선점

6.1 의의

해당 프로젝트를 통해 실제 증권 관련 서비스 중 리서치 페이지와 커뮤니티 페이지에 필요한 DB 스키마를 구현하고 실제로 사용할 수 있는 쿼리를 구현했습니다. 위와 같은 활동을 통해

실제 서비스에서 사용할 수 있는 데이터베이스를 구축하는 연습을 할 수 있었습니다.

6.2 개선점 및 향후 방향

짧은 시간으로 인해 대규모 데이터를 통한 테스트를 해보지 못했습니다. 따라서 대규모 데이터가 저장되었을 때 인덱스 설계나 쿼리 튜닝, 반정규화 등의 기능들이 필요하다고 생각합니다.

- **인덱스 설계** : 테이블 별로 사용되는 쿼리와 컬럼을 찾아서 인덱스 설계가 필요합니다.
- **쿼리 튜닝** : 현재는 기능 구현에 치중되어 있기 때문에 대규모 데이터를 통해 테스트를 하면서 느려지는 쿼리에 대해 쿼리 튜닝이 필요합니다.
- **반정규화** : 현재는 게시글의 댓글의 수를 조회하기 위해 COUNT 서브 쿼리를 사용하고 있지만, 반정규화를 통해 BOARDS 테이블에 REPLY_COUNT를 추가하는 등 반정규화를 고려해야 합니다.

7. 결론

본 프로젝트를 통해 금융권 리서치 서비스에 필요한 데이터베이스 설계와 실용적인 쿼리 구현을 성공적으로 완료했습니다. 키움증권과 토스증권의 실제 서비스를 참고하여 애널리스트의 리포트 관리와 사용자 상호작용을 위한 기능을 구현했으며, 이를 통해 실무에서 활용 가능한 데이터베이스 구축 경험을 쌓았습니다. 향후 대규모 데이터 처리를 위한 성능 최적화와 기능 개선을 통해 더욱 완성도 높은 서비스로 발전시킬 수 있을 것으로 기대됩니다.